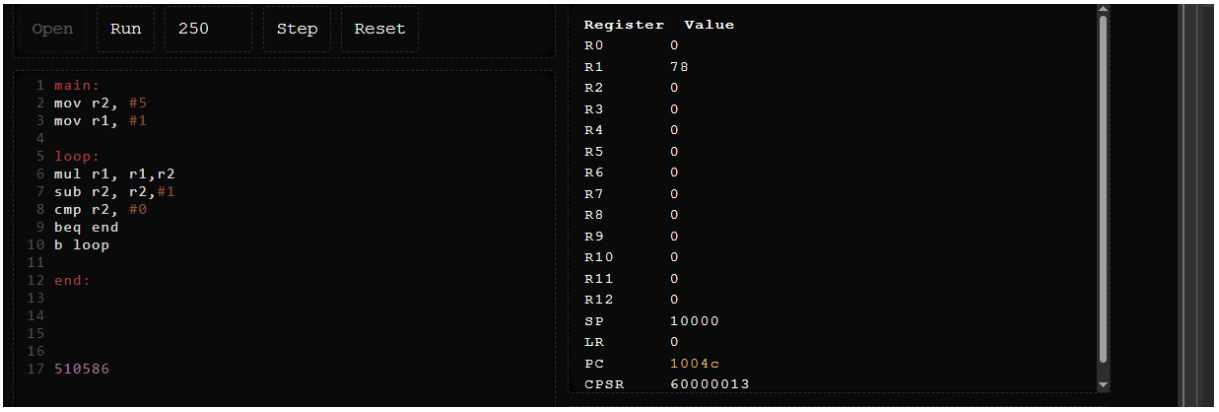
Template Week 4 – Software

Student number:510586

Assignment 4.1: ARM assembly

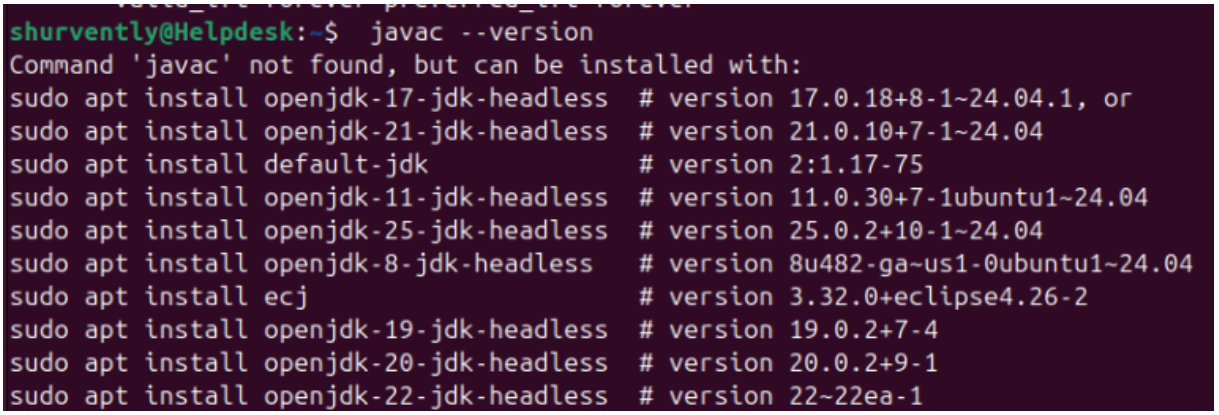
Screenshot of working assembly code of factorial calculation:



Assignment 4.2: Programming languages

Take screenshots that the following commands work:

javac --version



After sudo apt install default-jdk:

java --version

```
shurvently@Helpdesk:~$ java --version
Command 'java' not found, but can be installed with:
sudo apt install openjdk-17-jre-headless  # version 17.0.18+8-1~24.04.1, or
sudo apt install openjdk-21-jre-headless  # version 21.0.10+7-1~24.04
sudo apt install default-jre              # version 2:1.17-75
sudo apt install openjdk-11-jre-headless  # version 11.0.30+7-1ubuntu1~24.04
sudo apt install openjdk-25-jre-headless  # version 25.0.2+10-1~24.04
sudo apt install openjdk-8-jre-headless   # version 8u482-ga~us1-0ubuntu1~24.04
sudo apt install openjdk-19-jre-headless  # version 19.0.2+7-4
sudo apt install openjdk-20-jre-headless  # version 20.0.2+9-1
sudo apt install openjdk-22-jre-headless  # version 22~22ea-1
```

After sudo apt install default-jre:

```
shurvently@Helpdesk:~/Desktop/code$ java --version
openjdk 21.0.10 2026-01-20
OpenJDK Runtime Environment (build 21.0.10+7-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.10+7-Ubuntu-124.04, mixed mode, sharing)
```

gcc --version

```
shurvently@Helpdesk:~$ gcc --version
Command 'gcc' not found, but can be installed with:
sudo apt install gcc
```

After running sudo apt install gcc:

```
shurvently@Helpdesk:~/Desktop/code$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

python3 --version

```
shurvently@Helpdesk:~$ python3 --version

Python 3.12.3
```

bash --version

```
shurvently@Helpdesk:~$ bash --version
GNU bash, version 5.2.21(1)-release (x86_64-pc-linux-gnu)
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
```

Assignment 4.3: Compile

Which of the above files need to be compiled before you can run them?

- Fibonacci.java has to be compiled into bytecode before being executed
- Fib.c has to be compiled to machine code before being executed.
- fib.py needs to be interpreted, compiling isn't needed.
- fib.sh needs to be interpreted, compiling isn't needed.

Which source code files are compiled into machine code and then directly executable by a processor?

Only the Fib.c file gets compiled into machine code and is directly executable by the processor.

Which source code files are compiled to byte code?

Fibonacci.java is compiled to byte code.

Which source code files are interpreted by an interpreter?

Both fib.py and fib.sh are interpreted by an interpreter.

These source code files will perform the same calculation after compilation/interpretation. Which one is expected to do the calculation the fastest?

Fib.c is exected to run the fastest because it compiles into machine code which can be ran faster.

How do I run a Java program?

First it needs to be compiled and then it can be run.

```
shurvently@Helpdesk:~/Desktop/code$ javac Fibonacci.java
shurvently@Helpdesk:~/Desktop/code$ java Fibonacci
Fibonacci(18) = 2584
Execution time: 0.22 milliseconds
```

How do I run a Python program?

```
shurvently@Helpdesk:~/Desktop/code$ python3 fib.py
Fibonacci(18) = 2584
Execution time: 0.29 milliseconds
```

How do I run a C program?

First it needs to be compiled and then it can be run.

```
shurvently@Helpdesk:~/Desktop/code$ gcc fib.c -o fib
shurvently@Helpdesk:~/Desktop/code$ ./fib
Fibonacci(18) = 2584
Execution time: 0.01 milliseconds
```

How do I run a Bash script?

First you have to give the bash file permission by using the chmod command and then you can run it using sudo and the location of the folder/file.

Source:
https://www.hostinger.com/tutorials/how-to-run-sh-file-in-linux?utm_source=google&utm_medium=cpc&utm_id=23533378316&utm_campaign=Generic-Tutorials-DSA-t2|NT:Se|Lang:EN|LO:NL&utm_term=&utm_content={searchterm}&gad_source=1&gad_campaignid=23533378316&gbraid=0AAAAADMy-hYgoXS_vpIqtdpHxpHAxJk6x&gclid=Cj0KCQjwmunNBhDbARIsAOndKplQ5x8qpE6rU5qWy_gXsR9rKP8yKmVIlAPdJS9yOMeBRQVVeu-5LxEaAkSOEALw_wcB

```
shurvently@Helpdesk:~/Desktop/code$ sudo chmod a+x fib.sh
shurvently@Helpdesk:~/Desktop/code$ sudo ./fib.sh
Fibonacci(18) = 2584
Excution time 4592 milliseconds
```

If I compile the above source code, will a new file be created? If so, which file?

yes , when running javac fibonacci.java i get the new file called febonaccie.class

And from funning the C source code i get the fib file.

```
shurvently@Helpdesk:~/Desktop/code$ ls
fib  fib.c  Fibonacci.class  Fibonacci.java  fib.py  fib.sh  runall.sh
```

Take relevant screenshots of the following commands:

- Compile the source files where necessary
- Make them executable
- Run them
- Which (compiled) source code file performs the calculation the fastest?

Assignment 4.4: Optimize

Take relevant screenshots of the following commands:

a) **Figure out which parameters you need to pass to the gcc compiler so that the compiler performs a number of optimizations that will ensure that the compiled source code will run faster. Tip! The parameters are usually a letter followed by a number. Also read page 191 of your book, but find a better optimization in the man pages. Please note that Linux is case sensitive.**

```
-O
-O1 Optimize.  Optimizing compilation takes somewhat more time, and a lot more memory for a large function.

    With -O, the compiler tries to reduce code size and execution time, without performing any optimizations that take a great deal of compilation time.

    -O turns on the following optimization flags:

    -fauto-inc-dec  -fbranch-count-reg  -fcombine-stack-adjustments  -fcompare-elim  -fcprop-registers  -fdce  -fdefer-pop  -fdelayed-branch  -fdse  -fforward-propagate  -fguess-branch-probability  -fif-conversion
    -fif-conversion2  -finline-functions-called-once  -fipa-modref  -fipa-profile  -fipa-pure-const  -fipa-reference  -fipa-reference-addressable  -fmerge-constants  -fmove-loop-invariants  -fmove-loop-stores
    -fomit-frame-pointer  -freorder-blocks  -fshrink-wrap  -fshrink-wrap-separate  -fsplit-wide-types -fssa-backprop -fssa-phiopt -ftree-bit-ccp -ftree-ccp -ftree-ch -ftree-coalesce-vars -ftree-copy-prop -ftree-dce
    -ftree-dominator-opts -ftree-dse -ftree-forwprop -ftree-fre -ftree-phiprop -ftree-pta -ftree-scev-cprop -ftree-sink -ftree-slsr -ftree-sra -ftree-ter -funit-at-a-time

-O2 Optimize even more.  GCC performs nearly all supported optimizations that do not involve a space-speed tradeoff.  As compared to -O, this option increases both  compilation  time  and  the  performance  of  the
    generated code.

    -O2 turns on all optimization flags specified by -O1.  It also turns on the following optimization flags:

    -falign-functions  -falign-jumps  -falign-labels  -falign-loops  -fcaller-saves  -fcode-hoisting  -fcrossjumping  -fcse-follow-jumps  -fcse-skip-blocks  -fdelete-null-pointer-checks  -fdevirtualize
    -fdevirtualize-speculatively -fexpensive-optimizations -ffinite-loops -fgcse  -fgcse-lm -fhoist-adjacent-loads -finline-functions -finline-small-functions -findirect-inlining -fipa-bit-cp  -fipa-cp  -fipa-icf
    -fipa-ra  -fipa-sra  -fipa-vrp  -fisolate-erroneous-paths-dereference  -flra-remat  -foptimize-sibling-calls  -foptimize-strlen  -fpartial-inlining  -fpeephole2  -freorder-blocks-algorithm=stc
    -freorder-blocks-and-partition  -freorder-functions  -frerun-cse-after-loop  -fschedule-insns  -fschedule-insns2  -fsched-interblock  -fsched-spec  -fstore-merging  -fstrict-aliasing  -fthread-jumps
    -ftree-builtin-call-dce -ftree-loop-vectorize -ftree-pre -ftree-slp-vectorize -ftree-switch-conversion  -ftree-tail-merge -ftree-vrp -fvect-cost-model=very-cheap

    Please note the warning under -fgcse about invoking -O2 on programs that use computed gotos.

    NOTE: In Ubuntu 8.10 and later versions, -D_FORTIFY_SOURCE=2, in Ubuntu 24.04 and later versions, -D_FORTIFY_SOURCE=3, is set by default, and is activated when -O is set to 2 or higher.  This enables additional
    compile-time and run-time checks for several libc functions.  To disable, specify either -U_FORTIFY_SOURCE or -D_FORTIFY_SOURCE=0.

    NOTE: In Debian 13 and Ubuntu 24.04 and later versions, -D_TIME_BITS=64 together with -D_FILE_OFFSET_BITS=64 is set by default on the 32bit architectures armel, armhf, hppa, m68k, mips, mipsel, powerpc and sh4.

-O3 Optimize yet more.  -O3 turns on all optimizations specified by -O2 and also turns on the following optimization flags:

    -fgcse-after-reload  -fipa-cp-clone  -floop-interchange  -floop-unroll-and-jam  -fpeel-loops  -fpredictive-commoning  -fsplit-loops  -fsplit-paths  -ftree-loop-distribution  -ftree-partial-pre  -funswitch-loops
    -fvect-cost-model=dynamic -fversion-loops-for-strides
```

When looking at the man pages and searching for optimizations you get -O3 as the best optimization option.

The new command is: gcc -O3 fib.c -o fib

b) Compile **fib.c** again with the optimization parameters

```
shurvently@Helpdesk:~/Desktop/code$ gcc -O3 fib.c -o fib
```

c) Run the newly compiled program. Is it true that it now performs the calculation faster?

```
shurvently@Helpdesk:~/Desktop/code$ ./fib
Fibonacci(18) = 2584
Execution time: 0.00 milliseconds
```

Yes it does run faster.

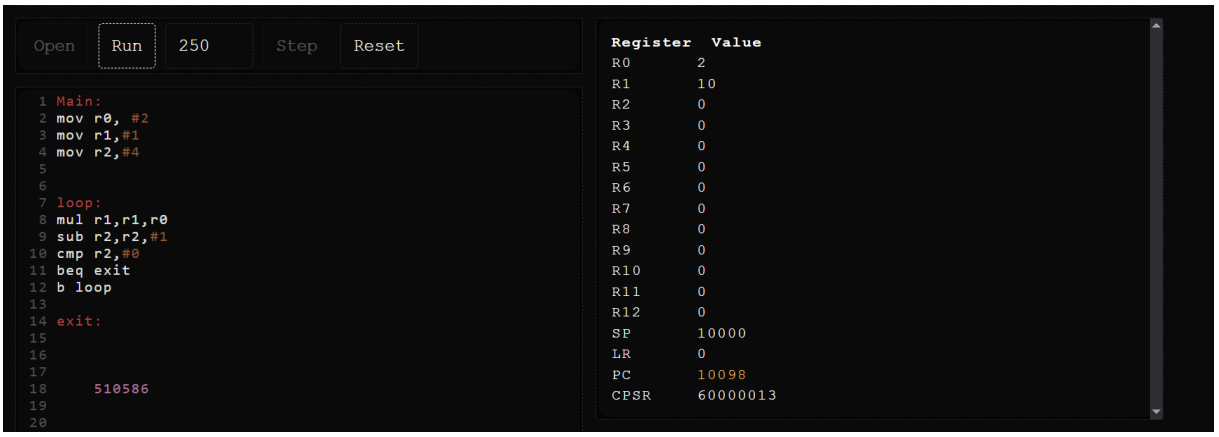
d) Edit the file **runall.sh**, so you can perform all four calculations in a row using this Bash script. So the (compiled/interpreted) C, Java, Python and Bash versions of Fibonacci one after the other.

```
Running C program:
Fibonacci(19) = 4181
Execution time: 0.01 milliseconds

real    0m0.001s
user    0m0.000s
sys     0m0.001s


Running Java program:
Fibonacci(19) = 4181
Execution time: 0.26 milliseconds

real    0m0.076s
user    0m0.036s
sys     0m0.059s


Running Python program:
Fibonacci(19) = 4181
Execution time: 0.49 milliseconds

real    0m0.013s
user    0m0.007s
sys     0m0.006s


Running BASH Script
Fibonacci(19) = 4181
Excution time 7441 milliseconds

real    0m7.444s
user    0m3.903s
sys     0m4.200s


shurvently@Helpdesk:~/Desktop/code$
```

*First you have to give the bash file permission by using the chmod command than i could run the bash file.

Assignment 4.5: More ARM Assembly

Like the factorial example, you can also implement the calculation of a power of 2 in assembly. For example you want to calculate $2^4 = 16$. Use iteration to calculate the result. Store the result in r0.




```
Main:
mov r1, #2
mov r2, #4

Loop:

End:
```

Complete the code. See the PowerPoint slides of week 4.

Screenshot of the completed code here.

Ready? Save this file and export it as a pdf file with the name: **week4.pdf**